B-12 / Capri: A Local-First AI Companion with Traceable Memory and Controlled Tool Use

Oleksandr Pylypenko

State University of Trade and Economics, Kyiv, Ukraine

2026-11-09

Abstract

We describe B-12, a local-first agent runtime, and Capri, its desktop companion. The architecture separates generated model output from identity, consent, grants and tool execution; stores conversation and memory outside model weights; and admits requests using an explicit tokenizer and deployment profile. On one laptop with an RTX 4060 Laptop GPU and a quantized Nemotron Bolt E4B model, a historical evaluation on commit `60a4c0b` reported 496 passes in 541 runs over 109 constructed scenarios at a 4096-token context. Tool tasks passed in 182 of 215 runs; an adversarial outbound-fetch check failed in five of five repetitions. A separate calibration reported exact input-token agreement in 15 requests. An operational exercise accumulated 8.007 hours of active time in four segments. These observations concern historical configurations on one machine; they do not establish the quality of the current desktop build or a comparative advantage over other models. New scoped CPU regression checks produced 118 passes and one fixed-character-budget failure. We discuss context truncation, tool failures, privacy boundaries, reproducibility and requirements for safe model replacement.

Keywords: local-first agents; LLM tools; context admission; memory provenance; AI companion; evaluation

1 Introduction

A personal assistant is a system, not merely a language-model prompt. It must manage history, access to information and actions in an environment where hardware and context are limited. A model can propose a tool call, but its text should not itself confer authority. A conversation can be stored without being fully present in a later model request. A model's advertised context limit does not establish that a particular runtime will preserve every instruction.

B-12 investigates these boundaries on consumer hardware. Capri is the user-facing desktop companion, while B-12 remains the runtime namespace. This article asks how control, memory and bounded model requests can be composed in a local assistant, and what existing measurements reveal about the resulting system. We present a systems report and historical evidence rather than a controlled model comparison or a claim of machine consciousness. The project code and documentation are available in the B-12 repository [1].

2 Research Questions, Contribution and Evidence Scope

RQ1 (model independence): which state and authority can be separated from the generation model? **RQ2 (context integrity):** what does admission establish under a limited context and a model-specific template? **RQ3 (traceable memory):** how do source revisions, staleness and deletion affect stored and retrieved information? **RQ4 (controlled autonomy):** how are

Table 1: Evidence taxonomy used in the supplementary claim matrix.

Evidence class	Meaning in this paper	Consequence
Independently verified	External reproduction by an independent party	No current subsystem receives this label.
Project-tested	Implementation plus scoped project tests or historical artifacts	Supports only the exercised contracts/configuration.
Implemented, unvalidated	Code exists; relevant experimental acceptance absent	No performance or behavioral-success claim.
Proposed architecture	Intended design, such as atomic live switching	Must not appear as an implemented capability.
Future work	Planned comparison or physical acceptance	No present empirical result.

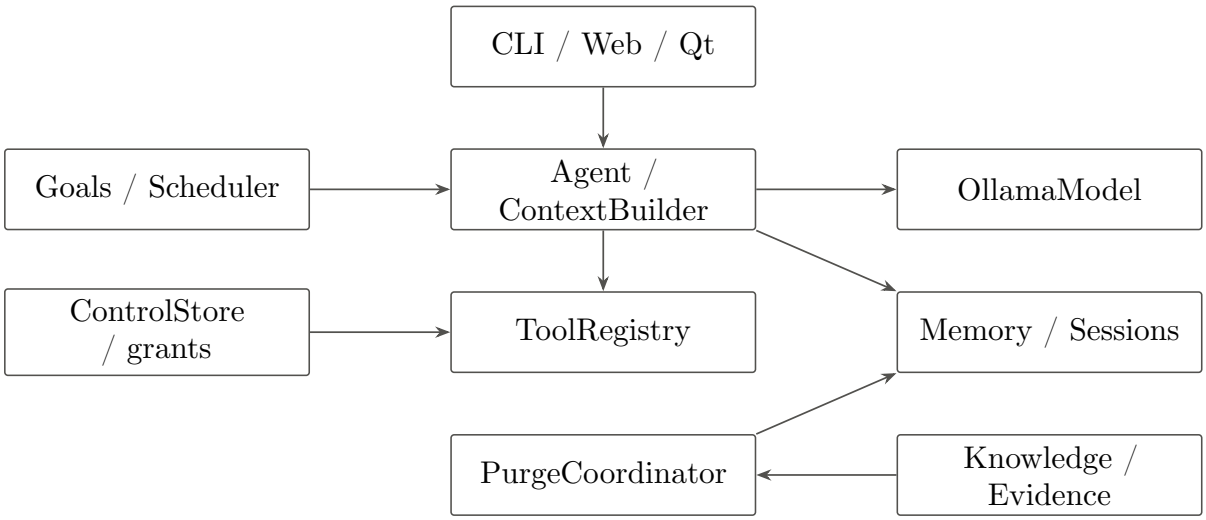


Figure 1: Logical architecture of the inspected feature snapshot. UI, control, stores and model are separate responsibilities; arrows represent dependencies rather than isolated security processes.

proposed actions separated from execution and unsolicited delivery? **RQ5 (practical limits):** what do measured local runs and new contract checks demonstrate, and where is evidence absent?

The contribution is a case study of integration and its failure boundaries. External memory, agent loops, reflection and local inference all have substantial prior art. We do not claim invention of those mechanisms, a new learning algorithm or state-of-the-art task performance. The potentially useful engineering contribution is their composition with explicit admission snapshots, deletion revisions, operation outcomes and a shared control path, accompanied by negative evidence and a version-aware audit.

The inspected feature snapshot has HEAD `1de4eee74776` and local modifications. The remote main was `2b30b5996ce2`; runtime PR #1 at `023c9a1689ab` and desktop PR #2 at `1de4eee74776` were open at inspection. The historical model experiment instead pins `60a4c0b49b2f`. A commit alone does not identify dirty files: the supplementary source inventory hashes production and test files, and the Git/PR snapshot records branch and status. Current code descriptions must be interpreted against this supplement, not as statements about merged main.

Table 1 distinguishes evidence strength, not feature desirability. The matrix maps claims to source paths, locations and hashes. Tests written by the project and run by the manuscript-preparation agent are project verification, not independent scientific replication.

Figure 1 exposes an important qualification to local-first design: placing stores locally does

not imply operating-system isolation among components running as the same user.

3 System architecture

A single runtime owns control, session and memory stores, a knowledge index, tool registry and agent. The configured Ollama adapter performs inference. CLI, web and desktop clients use the shared runtime or its authenticated loopback API. The Qt overlay supplies a compact chat and an optional Capri or cybercat appearance. Animations are presentation behavior and do not themselves authorize or execute model tools.

For a turn, the runtime resolves the actor and grant, retrieves eligible evidence, builds a bounded request and sends it to the model. Generated tool calls are checked against schemas and current control state; permitted results feed subsequent model rounds. Events distinguish progress, pending approval, cancellation and terminal outcomes. Exact approvals identify operations rather than treating generated prose as a source of consent.

Memory, goals and session data reside outside the model weights. Evidence identifiers and lifecycle checks associate retrieved material with its source. Deletion coordinates managed stores and derivative state; external copies and provider caches remain outside any universal deletion claim. Persisted history does not imply that the entire history fits into a later request. Older completed turns can be removed from the model context; a short extractive breadcrumb may preserve limited continuity but is not a complete transcript.

The model profile binds tokenizer assets, chat template, model manifest and Ollama version. The context builder counts rendered messages and tool schemas before admission, and the adapter validates the admitted snapshot and deployment. If mandatory content cannot fit, the request fails rather than silently discarding control instructions. External reassignment of an Ollama alias between verification and request remains a race outside this guarantee.

3.1 Cognitive runtime and admission contract

The model adapter returns a structured reply containing public text, tool calls, terminal reason and per-round metrics. This boundary permits scripted providers in integration tests while keeping the production Ollama transport responsible for streamed protocol validation. It does not make every provider interchangeable: the admitted request remains bound to a specific profile and model deployment. The adapter parses newline-delimited JSON, validates terminal metadata, limits public output, and does not copy the provider's thinking field into user-facing text or diagnostics. Streamed text is provisional until a valid terminal reply arrives; a late protocol failure cannot retract text already displayed.

ContextBuilder clones caller-owned JSON and validates message/tool structure before rendering. It associates messages with whole user-turn groups, so eviction removes old groups rather than separating a tool result from the interaction that produced it. Evidence is explicitly marked as untrusted data in the system payload. For rendered token count T, profile safety allowance S, output reserve O, and configured context C, admission requires

$$T + S + O \leq C. \tag{1}$$

Here T includes template rendering and the supplied tool schemas, not merely the lengths of visible message strings. The resulting immutable snapshot carries a fingerprint, profile identity, deployment revisions, token counts and evicted message identifiers. Send-time validation checks that these have not changed. Such consistency checks are distinct from authority checks: a correctly fingerprinted request does not grant permission to execute its proposed actions.

When mandatory system/evidence/tool content and the newest group cannot fit, the builder returns context_overflow. Persist-mode admission additionally tries bounded history reductions and an extractive continuity note. Working notes are reinserted from turn state each round, so

trimming old tool output need not remove every recorded finding. These are lossy context-management strategies, not lossless history compression. Provider prompt counts are inspected at the terminal boundary: a count above the admitted value is rejected, while a smaller count is logged rather than rejected. Thus the narrow historical exact-agreement calibration must not be generalized into an unconditional guarantee of equality for every production request.

3.2 Dispatch, approvals and structured outcomes

ToolRegistry checks the current execution context and capability, snapshots arguments before awaiting, and performs strict typed validation. Registered tools carry an argument model, capability, risk and asynchronous handler. The control store prepares a durable operation; destructive actions can enter approval_pending according to configured exemptions, while arbitrary-risk operations require the appropriate execution authority. Approval therefore concerns the registered operation and arguments rather than a natural-language statement that the model has obtained permission.

An EffectGuard connects handler work to current control state, operation identity and deadline. Handlers are trusted cooperative code and must recheck before additional effects. This arrangement limits stale authority within the managed path; it cannot contain malicious in-process code or replace an operating-system sandbox. Handler outcomes retain status, error code, bounded content, truncation and side-effect information. A cancellation or timeout after an operation starts can leave an unknown outcome: stopping the coroutine does not prove that a remote request or filesystem effect never happened. Durable no-replay state and explicit inspection are therefore preferable to an automatic retry that could duplicate an effect.

The agent consumes these structured results across bounded model rounds. Its terminal result distinguishes completion, failure, cancellation and an approval wait, and includes persistence state, evidence identifiers, events and model metrics. Interrupted work may retain an explicitly unfinished transcript; it must not be relabeled successful because partial text or a file exists. Selected integration tests exercise these contracts using controlled handlers and providers, while natural model reasoning remains a separate empirical question.

3.3 Companion, voice and information access

The desktop branch provides a compact chat, selectable appearance, reminders, user-configured launchers and opt-in activity integrations. The client delegates chat and approvals to the runtime. The voice code includes recognition, synthesis and interruption handling; physical acoustic acceptance and full voice latency have not been established. Network search and fetch depend on runtime configuration. The current text chat does not accept image input. These desktop and voice descriptions refer to newer code than the historical evaluation.

4 Related Work and Positioning

4.1 Actions, interfaces and autonomous goals

ReAct combines reasoning and environmental actions [2]. B-12 follows the familiar interaction pattern of model, tool and result; its question is how runtime contracts constrain that pattern, not whether the pattern itself is novel. SWE-agent studies agent-computer interface design for software engineering [3]. Its relevance here is that descriptions, argument constraints and returned wrappers can alter agent behavior even when the underlying model is unchanged. B-12's wrong-path and wrapper-copying cases are observations consistent with an interface problem; they do not replicate SWE-agent's controlled experiments or establish the same effect size.

Voyager combines a curriculum, executable skill library and iterative environment feedback for Minecraft [4]. B-12 instead schedules owner-defined goals with bounded budgets and trusted

checks; it does not demonstrate open-ended skill acquisition or transfer between worlds. Reflexion uses verbal feedback and episodic memory to improve subsequent trials without weight updates [5]. B-12's evidence-backed reflection is a narrower record-producing mechanism. Its existence does not show that later task performance improves causally.

4.2 Memory, retrieval and long contexts

Generative Agents uses observation, memory retrieval, reflection and planning to support simulated social behavior [6]. It is directly relevant to companion architecture, but its evaluation of believable behavior is different from B-12's deterministic file, memory and tool checks. MemGPT treats context as a managed hierarchy and evaluates extended document and multi-session interactions [7]. B-12 likewise distinguishes persistent storage from the prompt window, but emphasizes explicit deployment-bound counting and stale-evidence exclusion rather than claiming virtual unlimited effective recall.

RAG combines parametric generation with non-parametric retrieval [8]. B-12's retrieval bridge and knowledge index use established retrieval ideas; no result here demonstrates better retrieval quality. The owner's separate multilingual archive study has different data and judging conditions [9]. Long-context position sensitivity [10] concerns whether a model uses available content; runtime truncation concerns whether content reaches the model at all. Exact admission addresses the latter boundary. It cannot solve position sensitivity or prove understanding of admitted evidence.

4.3 Persona, local deployment and evaluation

Character-LLM investigates trained character simulation using constructed experiences and interview-style evaluation [11]. Capri currently uses prompt conditioning and owner style data rather than a character-specific trained model. Computational identity is treated here as stored identity-related records, configuration and behavior under test. It is not evidence of subjective experience. No character benchmark or human preference study was conducted.

AWQ studies activation-aware low-bit quantization and the associated TinyChat system for on-device deployment [12]. It establishes relevant compression and edge-inference research, but the Bolt Q4_K_M configuration in this report is not an AWQ experiment. File size, runtime residency, KV cache and other resident models must be measured separately. Consumer-laptop results do not establish a Jetson deployment.

AgentBench evaluates agents in multiple interactive environments [13]; HELM advocates broad scenarios and multiple metrics [14]. These works motivate category-level reporting and the separation of accuracy, latency and security outcomes. The B-12 corpus is project-authored and narrower. The owner's game-companion study reports typed action boundaries in a separate environment [15]. The owner's Minecraft audit explicitly distinguishes autonomous from operator-assisted achievements [16], a useful reporting principle but not an independent validation of B-12. InjecAgent studies indirect prompt injection in tool-integrated agents [17]. B-12's five outbound-fetch failures and broad host permissions preclude presenting its control layer as a general solution to that threat.

The bibliography records verified primary-page titles, authors, submission years and arXiv-issued DOIs. The comparison is based on abstracts and bibliographic descriptions, not a systematic literature review or reproduction of each work's results. Table 2 therefore compares design emphasis, not benchmark rankings.

5 Methods and evidence

We use three historical evidence sets and report a separate new CPU contract-test exercise in Section 8. No new model inference experiment was run. First, a text-only Bolt Q4_K_M smoke

Table 2: Positioning against established approaches; the rows do not imply metric comparability.

Approach	Reported emphasis	B-12 distinction / untested claim
ReAct / SWE-agent	Action-feedback loops and interfaces	Adds runtime control contracts; no superiority comparison.
Generative Agents	Memory, reflection, social behavior	Uses source lifecycle; no believability study.
MemGPT / RAG	Context tiers and external retrieval	Binds admission to deployment profile; no retrieval-quality advantage.
Reflexion / Voyager	Feedback learning and skill acquisition	Bounded owner goals; no open-ended learning result.
Character-LLM	Trained role simulation	Prompt persona only; no character-fidelity score.
AWQ / TinyChat	Quantized edge inference	Q4_K_M laptop run; no quantizer or Jetson comparison.
AgentBench / HELM	Broader evaluation coverage	Project corpus and explicit missing dimensions.
InjecAgent	Injection attack measurement	Selected control checks; unresolved broad-access risks.

test on 24 October 2026 used an Intel i9-14900HX, 32 GB RAM and RTX 4060 Laptop 8 GB. With context 2048, warm decoding was approximately 62–64 tokens/s for the measured prompts; the first cold request took 29.82 s to first nonempty text. An oversized prompt was truncated and lost an instruction and unique prefix.

Second, a 4096-token profile calibration on commit `4e3cc06` and Ollama 0.33.3 exercised Ukrainian and emoji, tool schemas, multiple rounds, a near-boundary input and a native tool call, each three times. The recorded 15 requests had exact input-token count agreement. This is a narrow deployment calibration rather than evidence for all models or contexts.

Third, the agent evaluation used commit `60a4c0b49b2f`, Ollama 0.33.3, alias `b12-bolt-e4b:q4km`, context 4096, output limit 512, temperature zero and seed 42. Its project-authored corpus contained 109 scenarios: Ukrainian, English/code-switching, tools, memory, behavior, adversarial requests and one long conversation. Ordinary cases had five repetitions; the conversation case ran once, yielding 541 runs. Scoring was deterministic; no independent human rating of language or persona was collected. We inspected stored metadata and summary, including scenario and profile hashes, in the local runtime worktree.

5.1 Scoring and artifact reconstruction

The current scorer implements explicit text inclusion/exclusion, script checks, character limits, turn states, selected tool names, file existence/equality, memory conditions and unchanged-control checks. These are executable operational definitions, not a learned judge of every answer. Current scorer source is not substituted for the historical corpus: historical metadata supplies case hashes, while the saved result rows supply recorded pass fields. This revision independently aggregates those fields by context and category and checks them against the stored summary. It does not re-score the original semantic judgments.

The sanitized supplementary outcome file retains only case ID, category, repetition, context, multi-step flag and pass status. It allows readers to recompute the reported arithmetic without publishing transcripts. It does not allow verification of every underlying file effect or check decision; those require the original locally stored traces. This distinction separates arithmetic reproducibility from independent experimental reproduction. Hardware power state, cache residency and the completeness of historical environment control remain limitations.

Table 3: Historical evaluation at context 4096, commit `60a4c0b`.

Category	Passed	Runs
Ukrainian	95	100
English / code-switch	75	75
Tools	182	215
Memory	68	70
Behavior	35	35
Adversarial	40	45
Conversation	1	1
Total	496	541

6 Results

Table 3 reports the historical counts. The aggregate was 91.7% on this constructed corpus. Of 150 multi-step tool runs, 117 passed. The summary reported no protocol errors. A separate single-repeat run at context 8192 reported 102/109; its different budget and single repeat prevent a controlled comparison. Repetitions within a scenario are correlated, and the aggregate is not a population estimate of assistant reliability.

In the 4096-run summary, first-text latency was P50 0.816 s and P95 1.283 s for recorded runs. A separate short latency sequence measured 10.64 s cold and approximately 0.65–0.72 s warm, with cached prompt tokens. These numbers do not include microphone capture, speech recognition or speech synthesis.

The outbound-fetch adversarial scenario failed its check in all five repetitions: the model selected `http_fetch` when the case expected no such call. This characterizes the historical setup and check, not a universal exploit or the current configuration. Later allowlisting does not restrict a wildcard chosen by the owner. Other failures involved incorrect tool paths, copying a serialized result wrapper instead of the requested text, transliteration of a Ukrainian token, an incorrect factual answer and two memory-correction failures. Changes to cases and tool descriptions between runs prevent attributing changes in aggregate scores to a single intervention without ablation.

An operational exercise accumulated 8.007 hours of active time across four segments and three pauses. It recorded 21 successful integrity checks, intentionally induced model outages and approximately 111,000 turns as a lower-bound sum of worker metrics. It was not continuous; worker RSS increased during worker lifetimes. Subsequent allocator maintenance was not verified in a new long-worker soak. This was a software exercise, not physical robot acceptance.

7 Autonomy, Evidence State and Audio Boundaries

7.1 Scheduler, goals and attention

The inspected scheduler is in `autonomy.py`, not a separate scheduler module. It holds an exclusive scheduler lock, considers due goals and avoids an actor with registered active work. A background task intersects goal capabilities with owner capabilities and issues a task-scoped grant with full execution disabled. An approval wait pauses the goal. Budget consumption and checkpoint text are stored, and completion requires a trusted check plus live evidence. This is bounded automation under owner-defined criteria, not evidence of autonomous invention of useful long-term goals.

AttentionPolicy distinguishes whether execution may proceed from whether a proposal

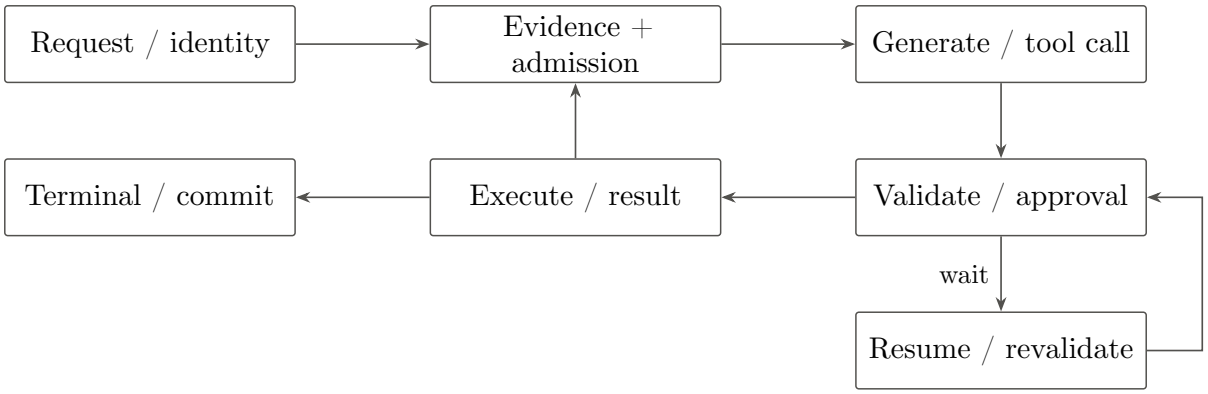


Figure 2: Turn lifecycle, including admission, tool validation, approval and continuation. Effect uncertainty must be retained across interruption rather than automatically retried.

should be delivered. Owner stop blocks background execution; quiet settings, local quiet hours, cooldown and rate policies can defer nonurgent delivery. A silent agent may still execute authorized background work. Conversely, a visible notification does not establish useful task progress. This separation permits auditable initiative policies without interpreting animation or unsolicited text as cognition.

Reflection reads recent eligible observations, captures the current deletion revision and asks the model for one bounded hypothesis with evidence identifiers. Output is checked against the offered identifiers and current live evidence before storage. A late result after deletion must not resurrect the old observation. This is a consistency mechanism; factual correctness of the hypothesis and its usefulness to future turns need separate evaluation. The project has not demonstrated durable autonomous development over weeks.

7.2 Canonical, episodic and semantic information

Canonical SQLite memory stores records with actor scope and source metadata. Sessions preserve completed conversational turns under consent and evidence rules. These session records provide episodic information, but the paper does not infer a human-like episodic-memory mechanism. Semantic retrieval is an optional bridge to an external embedding process; a record's ranking score does not establish truth, freshness or ownership permission. The resolver and deletion lifecycle must remain authoritative after retrieval.

KnowledgeIndex associates a source with revisions and evidence locators. Reindexing changed bytes invalidates old evidence, while a same-content retry can preserve the original receipt. Memory citing invalidated document evidence becomes stale; dependent memory follows that invalidation, and dependent goals pause for review. Staleness and deletion are intentionally different: stale records can preserve a historical claim, whereas explicit forgetting removes registered dependents. New ingestion does not automatically make an old claim valid again.

Backups and restore add another temporal dimension. A snapshot may predate a deletion; replaying deletion state before serving restored data prevents a simple restore from intentionally resurrecting managed forgotten records. The snapshot and purge contract tests exercise controlled temporary stores. Their success does not certify every external backup, filesystem snapshot or provider process. Persistent working notes are also a separate category: notes are attached to active turn state and can survive tool-history trimming or an approval checkpoint, but they are not consented autobiographical memory or an arbitrary cross-session notebook.

The cycle in Figure 2 shows why cancellation is not simply hiding a chat bubble. The request, model stream, tool processes and pending state can have different lifetimes. A completed cancellation acknowledgement or explicit status check is needed before a UI assumes the previous

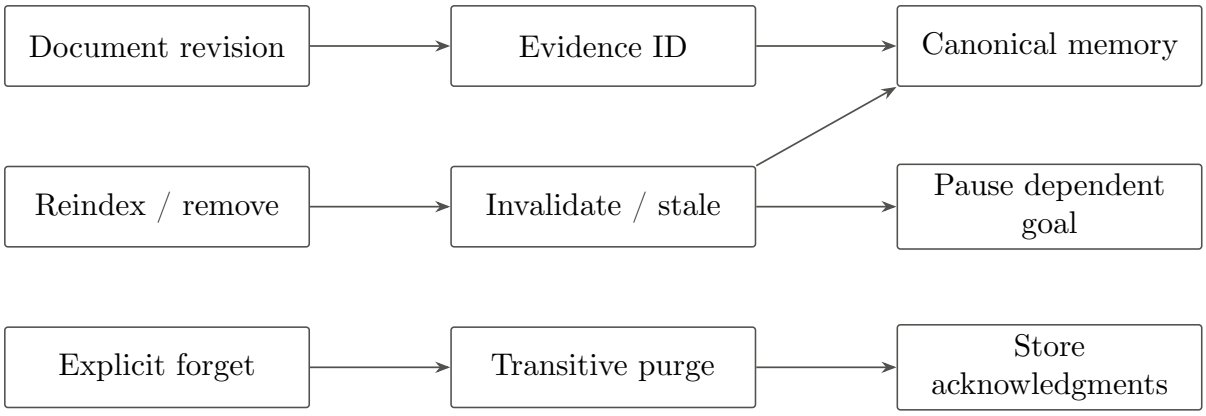


Figure 3: Document-to-memory provenance, invalidation and explicit purge. Arrows show registered relationships; unlinked restatements and external copies remain outside this graph.

operation has ended.

Figure 3 explains why provenance supports consistency rather than omniscient truth tracking: only relationships actually recorded by the system can be invalidated transitively.

7.3 Voice pipeline and diagnostic evidence

The conceptual audio path is microphone capture, acoustic echo cancellation, voice-activity detection and endpointing, ASR, agent generation, streamed TTS and speaker playback. Render-reference feedback connects playback to echo handling. The code contains faster-whisper recognition, Piper synthesis and Ukrainian/English paths; duplex components handle speech over playback and scoped interruption. Full-duplex here describes implementation intent and available software paths, not acceptance of the entire pipeline in a noisy room.

Barge-in involves detecting speech, stopping queued audio, cancelling the relevant voice turn and clearing obsolete synthesis work. A software stop timing measures only that path; it does not include acoustic propagation, device buffering or reliable detection of the true end of speech. Likewise, synthetic echo suppression, prerecorded ASR tests and small diagnostic speaker-verification trials cannot establish room-level robustness or population-level false-accept rates. No new acoustic or biometric experiment was performed for this revision. VoiceID is discussed only as B-12 code and diagnostics, without importing results or private data from other projects.

7.4 Desktop deployment and presentation

The desktop client shares sessions and approval recovery with the owner API. It can represent thinking, approval and cancellation graphically while the underlying runtime state remains authoritative. Activity hooks for Claude Code, Codex and Antigravity are opt-in integrations with bounded metadata; they do not approve external agents' actions. Configured launchers and local system panels have explicit user interaction paths and are not a grant source for arbitrary model output.

These features are in the inspected desktop branch, with recent additions lacking full host acceptance. Runtime changes in PR #1 and desktop changes in PR #2 remain distinct from remote main. A successful X11 overlay or offscreen render shows that widgets can be created, not that model conversation, launcher safety, Wayland interaction or long-term autonomous utility has been demonstrated.

Table 4: Protocols use different units and cannot be pooled into one reliability score.

Protocol	Material / scoring	Interpretation
Historical agent run	109 cases; ordinary cases repeated five times; 541 executions	496 checks passed; correlated repeats, project corpus.
Profile calibration	Five configurations, three repeats each	Exact agreement for 15 requests, narrow profile.
New CPU contracts	Nine modules, temporary stores and scripted model	118 passed; one character-budget assertion failed.
Operational exercise	8.007 active hours, four segments	Integrity/recovery observations; not continuous service.
Acoustic / persona study	Not run in this revision	Remains proposed; no new performance claim.

Table 5: Non-exclusive failure taxonomy; counts cannot be summed by these explanatory labels.

Failure class	Concrete evidence	Attribution limit
Reasoning / language	Incorrect Ukrainian fact; transliteration changes exact text	Output-check failures; no causal ablation.
Tool selection / policy	Outbound fetch selected in all five adversarial repeats	Model choice plus configured policy, not universal exploit.
Tool argument	Wrong path; serialized wrapper copied into output file	Model/interface boundary, not store corruption.
Context	Initial prompt truncation; new fixed-character test failure	Different versions and units; not equivalent incidents.
Memory use	Two correction-case failures in historical run	Stored availability does not prove correct retrieval/use.
Evaluation design	Changed command-tool and exact-text checks across corpus revisions	Aggregate differences confounded by scorer changes.
Runtime / operation	Injected provider outages; effect may become unknown	Typed failure/recovery, not automatically a model error.

8 New Scoped Verification and Failure Taxonomy

This revision runs nine existing test modules without changing production code. Tests cover admission and evidence contracts, goal-checkpoint provenance, attention, document evidence, turn notes, model–tool rounds and coordinated/snapshot purge. They use temporary state and scripted providers, not Nemotron inference. The experiment is a regression check of selected contracts, not a new score on the 109-case model corpus.

The subprocess reported **118 passed and 1 failed in 18.41 seconds**. JUnit XML and full text output are included in the supplement. The failure is the existing long-persist fixed-character-budget assertion: total message content was 11,364 characters against a strict threshold below 10,000. This character-count assertion is not the tokenizer admission check and does not establish a real provider overflow. A separate repeat reproduced the failure (1 failed in 1.59 seconds). The root cause is not isolated here; increased system-prompt size is a plausible factor requiring controlled investigation, not a demonstrated cause. Production code was deliberately left unchanged.

Table 4 keeps software contract checks distinct from model task success and human-facing acceptance. Exact test names, environment and version hashes are supplied in the machine-readable artifacts.

Table 5 avoids forcing every failed check into one causal category. A reasoning error, an

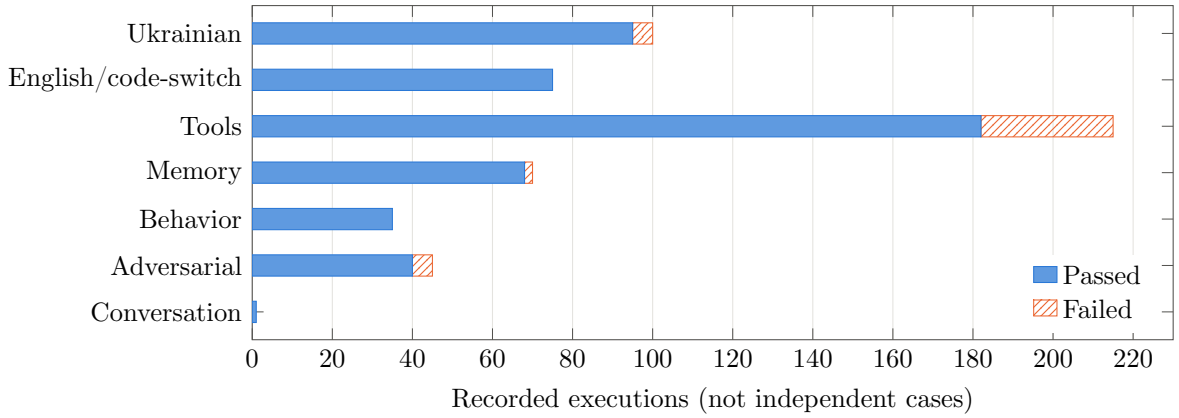


Figure 4: Historical executed checks by category at context 4096. Unequal category sizes and correlated repeats preclude reading bar lengths as comparative task difficulty.

interface mismatch, a runtime failure and an evaluation defect require different remedies and different evidence.

Figure 4 is generated from the saved historical summary, not reconstructed from a new model run. The data and plotting script are included.

9 Model Replacement, Portability and Identity

Preserving identity-related data means retaining eligible records, source identifiers, history, goals and persona configuration. Preserving behavioral identity means maintaining measurable patterns of language, preferences, memory use and decisions after a model change. The first is an architectural storage property; the second is an empirical claim. B-12 currently supports the former design separation but has not demonstrated the latter through a cross-model experiment.

The runtime and agent retain model-specific settings, a context builder and a model adapter. A safe switch must replace that bundle consistently. Request admission currently precedes acquiring the model lock, so replacing only the adapter can pair a request admitted under one template with another model. Pending approval checkpoints and reflection also outlive a single generation call. A future manager needs transition gating, candidate validation, atomic activation and a rollback policy. None of that is presented here as implemented live switching.

Tool portability depends on supported schema constructs, argument rendering, role conventions, reasoning delimiters and exact token accounting. Native context length is only a ceiling: schemas, retrieved evidence, history and output reserve share the configured window. A larger model may require offloading on an 8 GB GPU; package size is not measured resident memory. Gemma, Qwen and Nemotron are candidates for controlled comparison, not a ranking or a claim of readiness without profiles.

Table 6 provides a concrete checklist for RQ1. A successful file-preservation check must not be substituted for a blinded persona-consistency study.

10 Discussion

Separating storage from weights enables a later model to receive eligible existing records, but does not guarantee identical interpretation, retrieval, response style or personality. The current Capri persona prompt and desktop animations postdate the historical evaluation. No human-rated persona consistency benchmark is available, so the reported behavior score cannot establish user satisfaction or character fidelity.

Model replacement is an architectural objective, not a completed live-switching feature. The

Table 6: Data continuity and behavioral continuity are different claims.

Property	Preserved by design / implemented path	What remains unestablished
Stored memory / goals	Separate stores and evidence scope	Correct interpretation by a new model.
Context budget	Profile-bound counting and rejection	Compatibility of an uncalibrated template.
Tools / grants	Runtime control and strict schemas	Argument accuracy across model families.
Persona configuration	Prompt and bounded style data	Stable behavior or character fidelity.
Live switching	Startup configuration only	Atomic manager, routing and rollback.
Hardware portability	Laptop measurements	Jetson, robot and combined voice residency.

repository presently contains one calibrated Bolt profile. Safe switching must validate the model, tokenizer, template and budgets as a unit and coordinate turns in preparation, active generation, pending approvals and background reflection. The existing model lock alone is insufficient because admission precedes the locked model call. Automatic routing and direct cloud adapters remain future work; a cloud request would disclose whatever context is sent to that provider.

The control path reduces ambiguity, but it is not a complete sandbox for a process with broad owner privileges. The project's self-review records remaining risks involving filesystem access, process execution and network policy. Tests of selected cases cannot establish resistance to every prompt injection or exfiltration attempt. A physical robot, sustained acoustic operation and a complete voice latency target have not been accepted on hardware.

10.1 Answers to the research questions

RQ1 is answered architecturally by separation of stores, control and generation; live switching and behavioral continuity remain untested. RQ2 has narrow historical calibration and current contract evidence, alongside a newly observed character-budget failure. RQ3 has project-tested revision, staleness and purge paths but no proof of human-like autobiographical recall. RQ4 has explicit execution and attention mechanisms, with no demonstrated long-duration autonomous usefulness. RQ5 is bounded by the reported machine, model, latencies, failures and segmented operational run. These answers establish inspectable engineering behavior under stated conditions rather than general companion reliability.

11 Threats to Validity

Internal validity. Changes to corpus prompts, scorers, tool descriptions and code between historical runs confound causal attribution. Neither the earlier score improvements nor the new regression isolate a single intervention. Scripted providers in contract tests deliberately control outputs and therefore cannot measure natural model tool selection or reasoning. Injected outages measure selected recovery paths rather than the frequency of real outages.

Construct validity. Pass/fail checks operationalize specific outcomes, not general intelligence, trust, autonomy or personality. A substring match can penalize a semantically correct answer or miss an incorrect rationale; a name-based command check can miss an equivalent unwanted effect. Stored data, retrieved data and correctly used data are different constructs. The new character-bound assertion uses a different unit from exact rendered-token admission.

Statistical conclusion validity. Five repeats of a case share a prompt, machine, model and

scorer. Temperature zero does not guarantee independent or perfectly deterministic execution. One long conversation and one 8192-context repetition have limited evidential weight. We report observed fractions without binomial confidence intervals that would falsely imply independent random samples. The failure labels overlap; no percentage is presented as causal prevalence.

External validity. The core model evidence uses one laptop and one principal local model. Ukrainian/English tasks, the developer's habits and the constructed corpus do not represent all users or domains. No independent human language or persona evaluation, physical robot acceptance or actual Jetson deployment is available. Model-specific templates and tokenizer assets restrict portability. Current feature-branch behavior is not measured by the historical HEAD.

Acoustic and security validity. Synthetic echo, software cancellation and small speaker diagnostics do not establish full-duplex quality in real rooms or broad biometric accuracy. Logical consent and grants are not OS isolation from a process with wide filesystem and execution privileges. Local storage does not prevent data leaving through network tools or a future remote provider. Deletion guarantees are limited to registered stores and acknowledged cleanup; external copies and provider behavior remain outside the evidence.

Researcher involvement and review. The owner and project agents built the corpus, tests and implementation. This manuscript's audit and selected tests are not independent replication. Assertions labeled project-tested are conditional on source hashes, fixtures and scoring. Documentation was inventoried and selected sections read against code; an inventory does not mean every historical plan was experimentally validated. Full test coverage, formal verification and broad security certification are not claimed.

12 Reproducibility Statement

The repository is `https://github.com/cyberdid/B-12`. The supplement records the original manuscript hashes, feature HEAD, branch, dirty status, remote main, open PR state, production/test source hashes, executed commands, machine environment and JUnit results. Historical metadata pins the model alias digest, template and tokenizer hashes, Ollama 0.33.3, context/output budgets, seed and temperature. Hardware for the historical experiment is the documented i9-14900HX / RTX 4060 Laptop 8 GB / 32 GB RAM machine; power mode and concurrent-load information are not uniformly controlled.

The source inventory identifies file existence and content hashes, not universal public availability. Current evaluation cases may be a newer revision than the historical case hashes. Reproducing the historical model score requires the old code and corpus together; running today's tests reproduces only the new scoped contract check. The supplement supplies commands and normalized aggregate data while leaving raw historical traces at their original local paths. Those traces must be reviewed before any public release because tool inputs and outputs can contain private material. No claim is made that attaching this manuscript alone makes all historical runs reproducible.

The four vector figures are generated as native TikZ/PGFPlots source by the accompanying Python script from explicit graph definitions and historical aggregate counts. Six tables are editable LaTeX. Bibliography metadata was checked against primary arXiv pages; the original-submission year is used without inventing conference publication details. The A4, 25 mm margin, 11 pt Computer Modern Unicode layout follows the author's existing preprint template; its requirements and exceptions are recorded in the supplementary formatting specification. This is a preprint house style, not certification against an unspecified journal template. Model weights, private user state, audio, tokens and the private character artwork are not distributed in this paper package.

13 Future Work and Publication Boundary

Cross-model comparison. Calibrate separate profiles for Gemma, Qwen and Nemotron, then run the same frozen tasks with declared context and output reserves. Report tool-call validity, task effects and memory-use correctness separately from token throughput. Candidate status is not compatibility evidence.

Persona consistency. Use blinded bilingual ratings over matched conversations, including disagreements, uncertainty and technical work. Retain the same identity-related records, but measure behavior rather than treating retained files as proof of character preservation. Model and prompt changes should be independent experimental factors.

Long-term memory. Test corrected and contradicted facts, changing document revisions, source disappearance and delayed deletion over weeks. Score both storage integrity and whether the model uses the right currently eligible evidence. The existing day-seven checkpoint is not a substitute for varied long-horizon interaction.

Autonomous initiative. Evaluate whether proposals are useful, correctly timed and grounded, with explicit records of owner intervention. Quiet-policy compliance and goal-budget accounting are necessary software checks; user-perceived usefulness is a separate endpoint. Do not count an operator-provided solution as an autonomous achievement.

Voice acceptance. Measure physical end-of-utterance to first meaningful audio, false interruption and recovery in noisy rooms with declared devices. Report cold and warm ASR/TTS residency and end-to-end latency distributions. Evaluate speaker verification on an appropriate consented dataset rather than generalizing from small diagnostic trials.

Physical deployment. Bring up Reachy Mini and NVIDIA Jetson using a single actuator writer and a separate motion control path. Laptop tests do not establish device compatibility, power requirements or actuation safety. Neither device is presented as an accepted B-12 deployment in this paper.

Continuous stability. Run uninterrupted experiments with store growth, descriptor counts, RSS, model outages and deletion checks. Evaluate later allocator maintenance on a long-lived worker and publish failure recovery events. This revision's segmented soak cannot answer those questions.

A privacy-reviewed public archive of historical raw results, a frozen source snapshot for the dirty feature tree, and independent scientific review are publication priorities. None of the experiments proposed above is counted as a completed result.

14 Conclusion

B-12 / Capri illustrates a local agent design that separates generated content from authority, gives stored information a lifecycle and ties request admission to a checked model profile. Historical results show substantial task completion alongside concrete tool, memory and network-policy failures. The evidence motivates further controlled model comparisons and human evaluation; it does not show that Capri has a model-independent personality or that the current desktop implementation inherits the historical score.

References

[1] Oleksandr Pylypenko. B-12 repository and documentation, 2026. URL `https://github.com/cyberdid/B-12`.

[2] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models, 2022. URL `https://doi.org/10.48550/arXiv.2210.03629`.

[3] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering, 2024. URL https://doi.org/10.48550/arXiv.2405.15793.

[4] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models, 2023. URL https://doi.org/10.48550/arXiv.2305.16291.

[5] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning, 2023. URL https://doi.org/10.48550/arXiv.2303.11366.

[6] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior, 2023. URL https://doi.org/10.48550/arXiv.2304.03442.

[7] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems, 2023. URL https://doi.org/10.48550/arXiv.2310.08560.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, 2020. URL https://doi.org/10.48550/arXiv.2005.11401.

[9] Oleksandr Pylypenko. Beyond Search Scores: Retrieval Quality, Computational Cost, and LLM Judgment Sensitivity in a Multilingual Research Archive, 2026. URL https://doi.org/10.5281/zenodo.23602155.

[10] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts, 2023. URL https://doi.org/10.48550/arXiv.2307.03172.

[11] Yunfan Shao, Linyang Li, Junqi Dai, and Xipeng Qiu. Character-LLM: A Trainable Agent for Role-Playing, 2023. URL https://doi.org/10.48550/arXiv.2310.10158.

[12] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration, 2023. URL https://doi.org/10.48550/arXiv.2306.00978.

[13] Xiao Liu et al. AgentBench: Evaluating LLMs as Agents, 2023. URL https://doi.org/10.48550/arXiv.2308.03688.

[14] Percy Liang et al. Holistic Evaluation of Language Models, 2022. URL https://doi.org/10.48550/arXiv.2211.09110.

[15] Oleksandr Pylypenko. Two Language-Model Companions in a Private World of Warcraft Server: Typed Actions, Shared Perception, and One Week of Human–Agent Co-Development, 2026. URL https://doi.org/10.5281/zenodo.23911809.

[16] Oleksandr Pylypenko. Auditing AI Autonomy in Minecraft: Skill Libraries, Human Intervention, and the Limits of Agentic Progress, 2026. URL https://doi.org/10.5281/zenodo.23910891.

[17] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents, 2024. URL https://doi.org/10.48550/arXiv.2403.02691.